Full-Stack Engineering Training — Day 11 Study Material

MODULE	TOPIC	PORT & STACK
Backend Foundations (NestJS)	Architecture, Modules, Controllers & CRUD	Port 3011 • Express Engine • TypeScript

Key Learning Objectives: Understand the enterprise motivation behind NestJS compared to unopinionated frameworks like Express, master the core three-tier architectural hierarchy (`Modules` , `Controllers` , `Services`), learn Dependency Injection (DI) through constructor injection, implement complete in-memory CRUD endpoints with HTTP status codes, and model hierarchical nested resources (`/projects/:id/towers`).

1. Mental Model: Why NestJS Over Plain Express?

Plain Express or Flask provides routing but imposes zero architectural structure. In multi-developer teams, this leads to messy, inconsistent codebases with tightly coupled business logic and untestable endpoints.

The NestJS Solution:
- Modular organization grouping related code into cohesive domains (`ProjectsModule` , `TowersModule`).
- Built-in Inversion of Control (IoC) container managing service lifecycles.
- First-class TypeScript support with declarative decorators for routes, parameters, and headers.

2. The NestJS Architecture Triangle

Every feature in NestJS is structured across three core layers:

Layer	Decorator	Primary Responsibility
Module	`@Module()`	Defines boundaries, binds controllers and providers, exports shared capabilities
Controller	`@Controller()`	Handles incoming HTTP requests, route mapping, request body extraction, and responses
Provider (Service)	`@Injectable()`	Contains all core business logic, calculations, and data persistence operations

3. Dependency Injection (DI) & Constructor Injection

Instead of manually instantiating classes with `new ProjectsService()`, NestJS's IoC container automatically resolves and provides the singleton service instance via constructor injection:

```
// projects/projects.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';

@Injectable()
export class ProjectsService {
  private projects = [
    { id: 'proj-1', name: 'Skyline Commercial Park', status: 'ACTIVE' }
  ];

  findAll() { return this.projects; }

  findById(id: string) {
    const item = this.projects.find(p => p.id === id);
    if (!item) throw new NotFoundException(`Project ${id} not found`);
    return item;
  }
}

// projects/projects.controller.ts
import { Controller, Get, Param } from '@nestjs/common';
import { ProjectsService } from './projects.service';

@Controller('projects')
export class ProjectsController {
  // ProjectsService is automatically injected by NestJS!
  constructor(private readonly projectsService: ProjectsService) {}

  @Get()
  getAll() {
    return this.projectsService.findAll();
  }

  @Get(':id')
  getOne(@Param('id') id: string) {
    return this.projectsService.findById(id);
  }
}
```

4. Nested Resources Modeling (/projects/:id/towers)

In construction project models, towers belong to projects. We model this cleanly using parameterized controller paths:

```
@Controller('projects/:projectId/towers')
export class TowersController {
  constructor(private readonly towersService: TowersService) {}

  @Get()
  getTowers(@Param('projectId') projectId: string) {
    return this.towersService.findByProject(projectId);
  }

  @Post()
  createTower(@Param('projectId') projectId: string, @Body() data: any) {
    return this.towersService.create(projectId, data);
  }
}
```

5. Common Beginner Pitfalls & "Intern Watch-Outs"

> **1. Forgetting to Register Providers in the Module:**
> If you create a new Service but forget to add it to the `providers: [...]` array of your module, NestJS will crash with a dependency resolution error on application startup.

> **2. Putting Business Logic Inside Controllers:**
> Controllers must stay lean. Their sole job is to receive HTTP requests and return responses. Never write database logic or complex calculation loops in controllers.

> **3. Forgetting Module Exports for Shared Services:**
> If `TowersModule` needs `ProjectsService`, `ProjectsModule` must explicitly add `exports: [ProjectsService]`.

6. Summary & Memory Cheat Sheet

- **M-C-S Hierarchy**: **M**odule (Envelope) → **C**ontroller (Traffic Officer) → **S**ervice (Worker).
- **HTTP Status Norms**: `200 OK` for GET/PUT, `201 Created` for POST, `404` for missing resources.
- **Dependency Injection**: Declare your dependency in the constructor; let NestJS manage its lifecycle.

7. Verification & Exercise Output (o/p)

```
--- RUNNING DAY 11 NESTJS ARCHITECTURE & CRUD TESTS ---
✓ Test 1: GET /projects returned all initial projects (Status 200)
✓ Test 2: GET /projects/:id retrieved single project (Status 200)
✓ Test 3: POST /projects successfully created project (Status 201)
✓ Test 4: PUT /projects/:id successfully updated project (Status 200)
✓ Test 5: DELETE /projects/:id removed project and throws 404 (Status 200/404)
✓ Test 6: GET /projects/:id/towers returned nested towers (Status 200)
✓ Test 7: POST /projects/:id/towers added nested tower resource (Status 201)
ALL 7 ENDPOINT TESTS PASSED SUCCESSFULLY!
```